

Introduction to Computer Organisation and Architecture UCCD1133



Chapter 4

Computer Architecture and Organization Fundamental

Disclaimer



- This slide may contain copyrighted material of which has not been specifically authorized by the copyright owner. The use of copyrighted materials are solely for educational purpose. If you wish to use this copyrighted material for other purposes, you must first obtain permission from the original copyright owner.

Outline

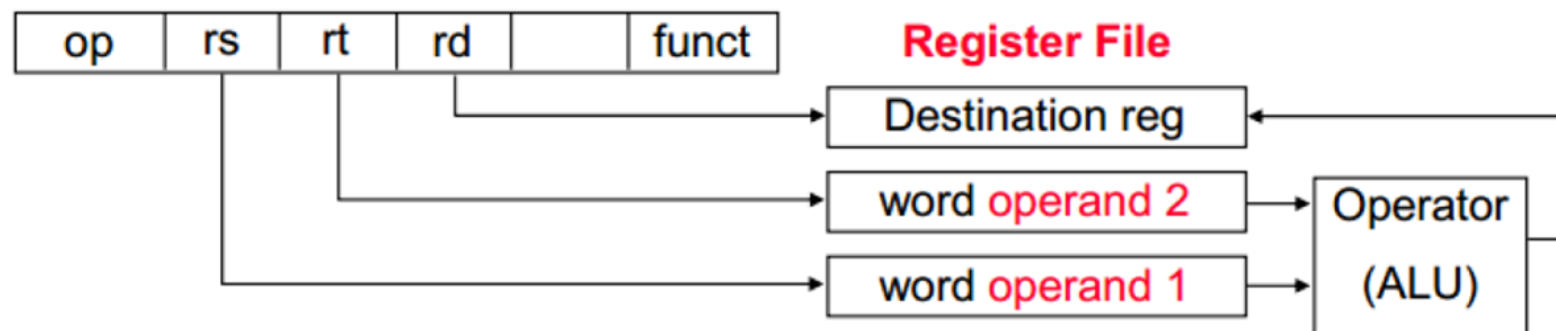
- Addressing Modes
- The Memory Map

Addressing Modes

- What is addressing mode?
 - Addressing modes are the ways of specifying an operand or a memory address.
 - The address of the operand determined is called the effective address.
- Basic MIPS addressing modes:
 - Register addressing
 - Immediate addressing
 - Base (displacement) addressing
 - PC-Relative addressing
 - Pseudo-direct addressing

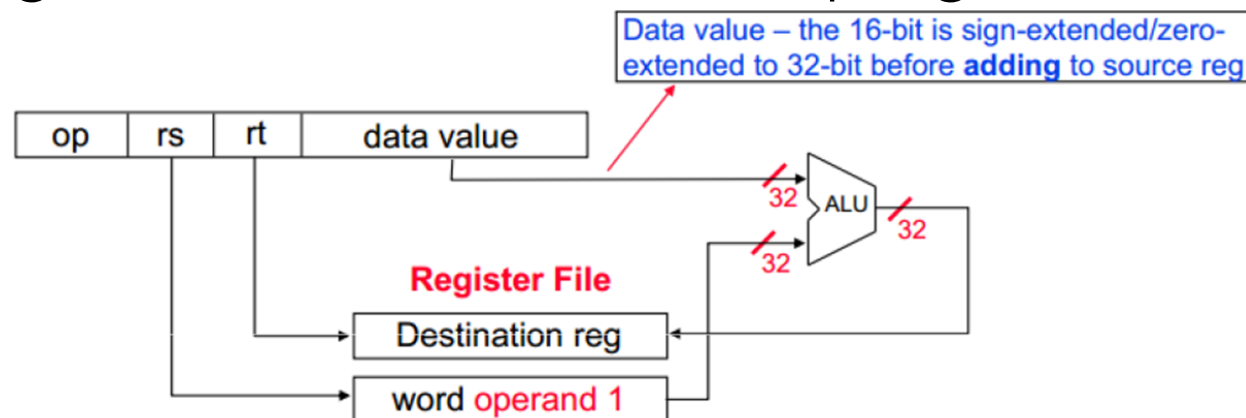
Register Addressing

- The operand is in a register.
 - Example: add \$s0, \$t2, \$t3
 - Example: sub \$t8, \$s1, \$0
- Used by instructions:
 - Arithmetic instructions: add, sub, ...
 - Logical instructions: and, or, sll, srl, ...
 - Program control instructions: slt, sltu, jr, ...



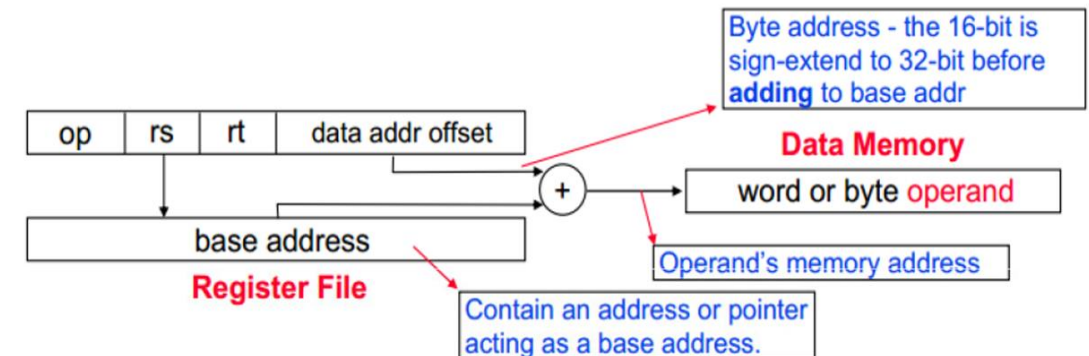
Immediate Addressing

- The operand is a 16-bit constant contained within the instruction.
 - Example: `ori $t3, $t7, 0x00FF`
- Used by instructions:
 - Arithmetic and logical instructions: `addi`, `andi`, `ori`, ...
 - Data transfer instructions: `lui`
 - Program control instructions: `slti`, `sltiu`, ...
- Zero-extending 16-bit – logical immediate instructions (`andi`, `ori`, etc.)
- Sign-extending 16 bit – arithmetic and program control immediate instructions (`addi`, `addiu`,



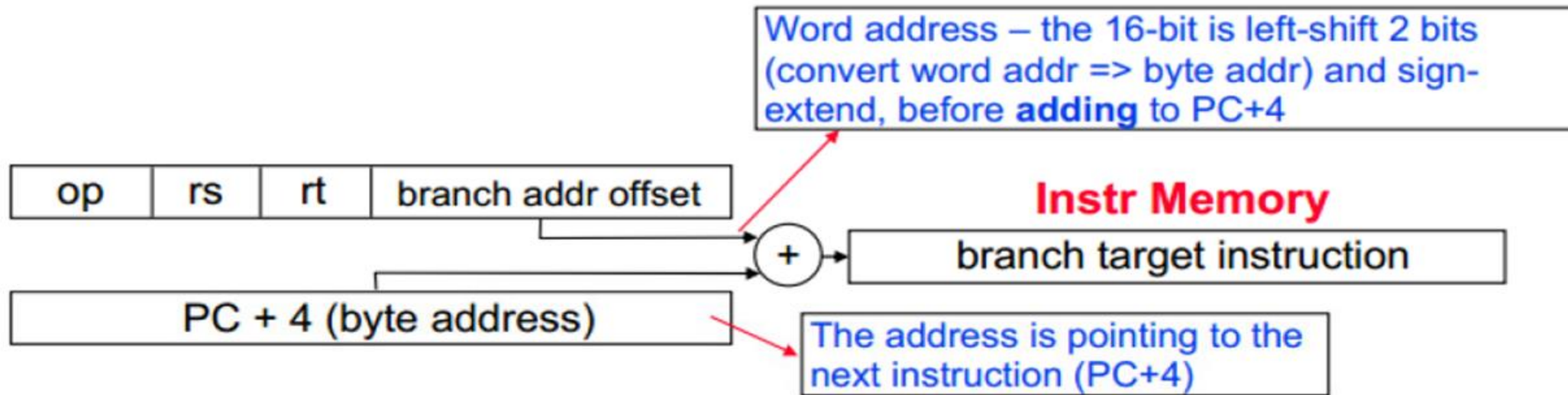
Base (displacement) Addressing

- The operand's memory address = base address + sign-extended immediate
 - Example: lw \$s4, 72(\$0)
 - address = \$0 + 72
 - Example: sw \$t2, -24(\$t1)
 - address = \$t1 - 24
- Used by instructions:
 - Data transfer instructions: lw, sw, lb, sb



PC-Relative Addressing

- The new instruction's memory address is obtained by summing the PC and a 16-bit constant contained within the current instruction.
- Used by instructions:
 - Branch instructions: beq, bne



PC-Relative Addressing

Example:

```

0x10      beq    $t0, $0, else
0x14      addi   $v0, $0, 1
0x18      addi   $sp, $sp, i
0x1C      jr     $ra
0x20      else:  addi   $a0, $a0, -1
0x24      jal    factorial
  
```

Solution

Assembly Code

Field Values

beq \$t0, \$0, else

op	rs	rt	imm
4	8	0	3
6 bits	5 bits	5 bits	16 bits

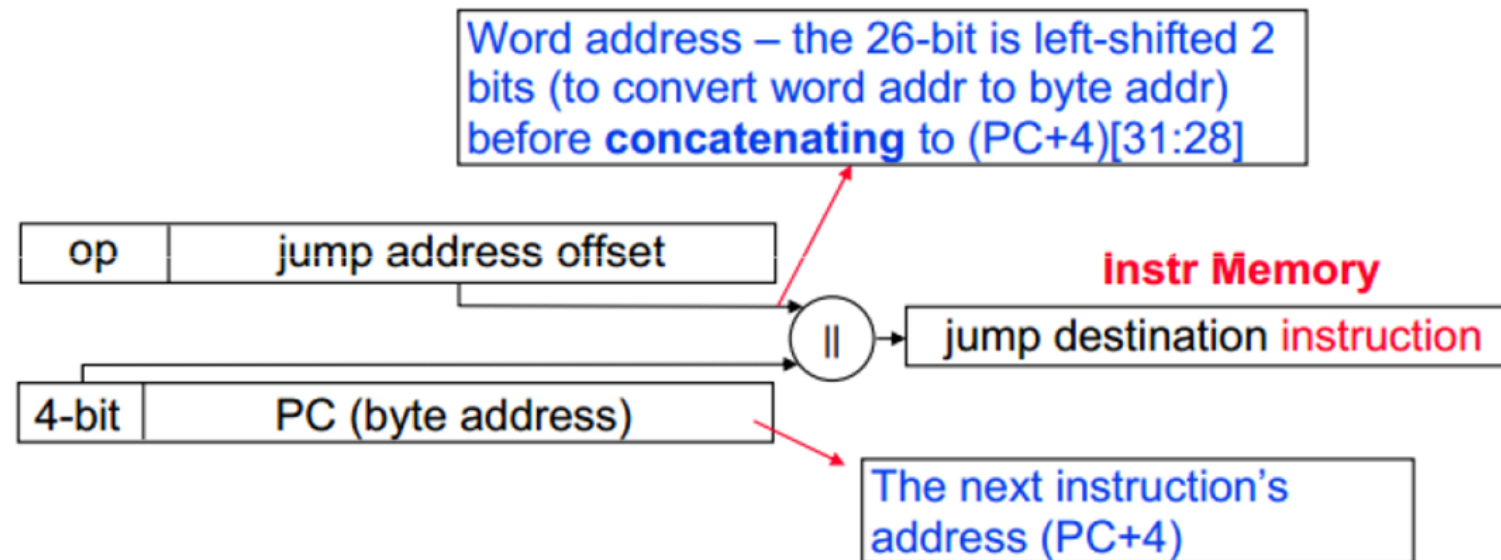
Machine Code

op	rs	rt	imm
000100	01000	00000	000000000000000011
6 bits	5 bits	5 bits	16 bits

(0x11000003)

Pseudo-direct Addressing

- The new instruction's address (jump target address) in memory is obtained by concatenating the left-shifted 26-bit constant contained within the current instruction with the upper 4-bits of the PC.
- Used by instructions:
 - Jump instructions: j, jal.



Pseudo-direct Addressing

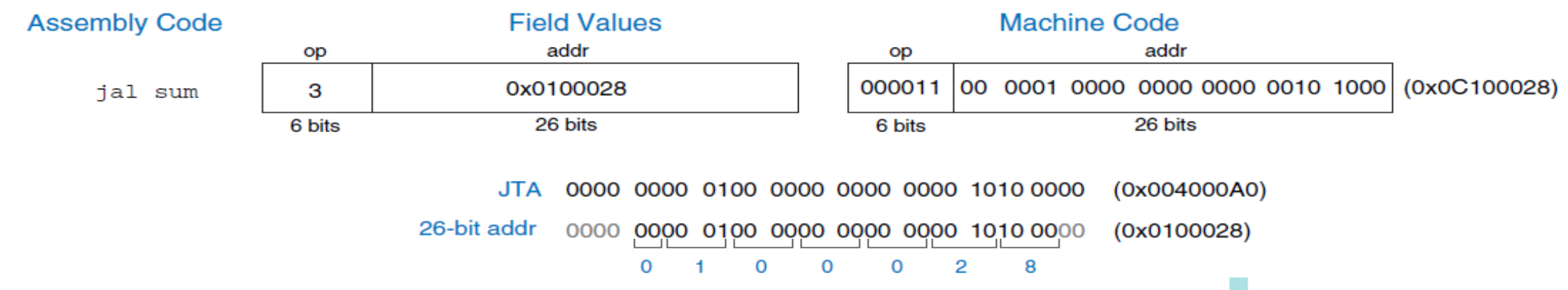
Example

MIPS Assembly Code

```

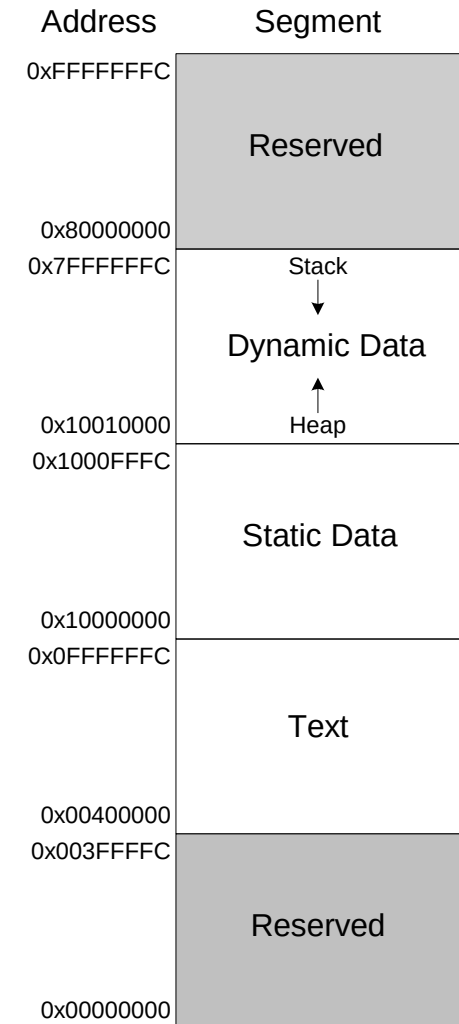
0x0040005C      jal      sum
...
0x004000A0  sum:  add     $v0, $a0, $a1
  
```

Solution



MIPS Memory Map

- What is stored in memory?
 - Instructions (also called text)
 - Data
 - Global/static: allocated before program begins
 - Dynamic: allocated within program
- How big is memory?
 - At most $2^{32} = 4$ gigabytes (4 GB)
 - From address 0x00000000 to 0xFFFFFFFF



Q & A

